AEM Interview Questions – Complete Answers

Adobe Experience Manager – 49 Questions Covered

Section A – Questions from your notebook (25)

Q1. AEM 6.5 vs AEM as a Cloud Service (AEMaaCS)

- **Hosting:** AEM 6.5 is on-prem / IaaS. AEMaaCS is fully managed by Adobe on cloud.
- **Versioning:** 6.5 has fixed releases & service packs. AEMaaCS is always up-to-date (continuous delivery).
- **Author/Publish:** 6.5 has fixed instances. AEMaaCS uses auto-scaling, ephemeral pods.
- **Code deployment:** 6.5 uses CRX Package Manager / Maven build. AEMaaCS uses Cloud Manager pipelines only.
- **Dispatcher:** 6.5 uses traditional config. AEMaaCS uses Dispatcher SDK with stricter rules.
- **Assets:** 6.5 uses on-prem DAM. AEMaaCS uses Asset microservices (cloud processing).
- **Customization:** 6.5 allows OSGi runtime config changes. AEMaaCS only allows config via /apps (immutable).
- **Repository:** 6.5 has /apps and /content writable. AEMaaCS – /apps is immutable, only /content & /conf are mutable.

Q2. How to create a component in AEM

Steps:

- In CRXDE Lite, navigate to /apps/<project>/components.
- Right-click → Create → Create Component.
- Provide Label, Title, Group, Super Type (e.g., core/wcm/components/text/v2/text).
- This creates a node of type cq:Component with a .html (HTL) file of the same name.
- Add a cq:dialog (nt:unstructured, sling:resourceType=cq/gui/components/authoring/dialog) for authoring fields.
- Optionally add Sling Model (Java class with @Model annotation) to expose business logic.
- Add component to allowed components list of the template policy so authors can drop it on a page.

Q3. Main features of AEM

- Content Management System (WCM) – author once, publish anywhere.
- Digital Asset Management (DAM) – central repository for images, videos, docs.
- Multi-Site Manager (MSM) – manage multiple sites with shared content.
- Workflows – automated content approval / processing.
- Personalization & Targeting (with Adobe Target).
- Translation framework for multilingual sites.
- Experience Fragments and Content Fragments for headless / omnichannel delivery.
- Integration with the Adobe Experience Cloud (Analytics, Campaign, Target).

Q4. Why AEM is used

AEM is used because it provides an enterprise platform that unifies content creation, asset management, personalization and delivery. Key reasons:

- Single platform to build, manage and deliver web content and assets.
- Built-in versioning, workflow and rollback features.
- Easy multi-site, multi-language management.
- Tight integration with Adobe Marketing Cloud.
- Robust permission model with users / groups / ACLs.
- Headless and hybrid delivery via Content Fragments and GraphQL APIs.

Q5. Difference between AEM Author and Publish instance

- **Author:** Used by content authors to create, edit, review content. Has authoring UI, workflows, sidekick. Usually inside corporate network.
- **Publish:** Serves content to end users. Read-optimized, no authoring UI. Sits in DMZ behind dispatcher.
- **Replication:** Content is moved from Author → Publish via replication agents.
- **Ports:** Default Author 4502, Publish 4503.

Q6. JCR (Java Content Repository)

JCR is a Java specification (JSR-170/283) defining a standard API to access content repositories. AEM uses Apache Jackrabbit Oak as its JCR implementation.

- Hierarchical / tree-based storage (everything is a node).
- Supports versioning, observation, search (XPath, JCR-SQL2), transactions, locking, ACLs.
- Each node has a primary node type (cq:Page, nt:unstructured, etc.) and properties.
- Accessed in AEM via Session, Node, Property, Resource (Sling abstraction).

Q7. Where is content stored? → under /apps?

Common misconception. The correct mapping is:

- **/content:** Actual page content authored by users (this is where pages live).
- **/content/dam:** Digital assets (images, videos, PDFs).
- **/apps:** Project code – components, templates, clientlibs, configs (immutable in AEMaaCS).
- **/libs:** Adobe-provided code – never modify, only overlay.
- **/conf:** Editable templates, policies, context-aware configurations.
- **/etc:** Legacy configurations (mostly deprecated, moved to /conf and /apps).
- **/var:** Runtime data – workflows, audit logs.

Q8. Editable Template

An editable template (introduced in AEM 6.2) allows authors to define the structure, initial content, layout and policies of pages through the UI – no developer intervention needed for layout changes.

- Lives under /conf/<project>/settings/wcm/templates/.
- Three modes: Structure, Initial Content, Layout.

- Linked to a page through cq:template property.
- Maintains live connection – changes to structure are inherited by existing pages.

Q9. What components are fixed in editable templates? (Header / Footer)

In an editable template, in Structure mode, the template author can lock certain components so that page authors cannot remove or edit them. Typical locked (fixed) components are:

- Header (logo, navigation)
- Footer (legal links, copyright)
- Sometimes the main page container or breadcrumb.

These are configured in Structure mode by clicking the lock icon. Unlocked components become an 'unlocked area' (parsys/responsive grid) where page authors can add content.

Q10. What is cq:dialog

cq:dialog is a JCR node (of type nt:unstructured with sling:resourceType=cq/gui/components/authoring/dialog) that defines the Touch UI authoring dialog for a component.

- Lives inside a component as /apps/<project>/components/<comp>/_cq_dialog/.content.xml.
- Defines fields (textfield, pathfield, image, multifield, etc.) authors fill in.
- Built using Granite UI / Coral UI resource types.
- Saved as properties on the component's resource node under the page (jcr:content/<component>).

Q11. Have you created a component using sling:resourceSuperType?

Yes. Example – extending the core title component:

- Create node /apps/myproj/components/title.
- Set sling:resourceSuperType="core/wcm/components/title/v3/title".
- It inherits everything (HTL, dialog, model).
- You can override only what you need – e.g., add custom HTL, extend the dialog by referencing the parent via sling:resourceSuperType inside _cq_dialog.

This is the recommended pattern: never copy core components, always inherit via sling:resourceSuperType (the 'proxy pattern').

Q12. In template, do we use sling:resourceType or sling:resourceSuperType?

In a template (under /conf/.../structure/jcr:content/root/...) we use sling:resourceType – it points to the actual component to render.

sling:resourceSuperType is used inside a component definition (under /apps) to declare inheritance from another component – it's a developer-side relationship, not a content-side one.

- **sling:resourceType:** What component renders this resource (used in content / templates).
- **sling:resourceSuperType:** Which component does this component inherit from (used in /apps).

Q13. Have you created any page?

Yes. Steps to create a page programmatically or manually:

- Sites Console → select parent page → Create → Page.
- Select a template (from /conf/<project>/settings/wcm/templates).
- Enter title, name, tags.
- AEM creates a cq:Page node with a jcr:content child of type cq:PageContent.
- Programmatically: use PageManager#create(parentPath, name, templatePath, title).

Q14. Data-sly keywords – briefly explain

- **data-sly-use:** Initializes a Sling Model or Use-class. e.g., data-sly-use.bean="com.proj.Model".
- **data-sly-test:** Conditional rendering. e.g., data-sly-test="${bean.show}".
- **data-sly-list:** Iterates over a collection. e.g., data-sly-list.item="${bean.items}".
- **data-sly-repeat:** Same as list but repeats the element itself, not just children.
- **data-sly-include:** Includes another HTL file. e.g., data-sly-include="partial.html".
- **data-sly-resource:** Includes another resource (component). e.g., data-sly-resource="${ 'header' @ resourceType='proj/components/header' }".
- **data-sly-template / data-sly-call:** Define and call reusable HTL templates.
- **data-sly-attribute:** Dynamically sets HTML attributes.
- **data-sly-element:** Dynamically sets the HTML tag name.
- **data-sly-unwrap:** Removes the host element while keeping its children.
- **data-sly-set:** Creates a local variable.

Q15. Have you created a Sling Model?

Yes. A Sling Model is a POJO that adapts from a Resource or SlingHttpServletRequest and exposes data to HTL.

```
@Model(adaptables = Resource.class,        defaultInjectionStrategy =
DefaultInjectionStrategy.OPTIONAL) public class TitleModel {      @ValueMapValue
private String title;      @ValueMapValue(name = "jcr:description")     private String
description;      @PostConstruct     protected void init() { /* logic */ }      public
String getTitle() { return title; }     public String getDescription() { return
description; } }
```

In HTL: data-sly-use.model="com.proj.models.TitleModel" then ${model.title}.

Q16. Annotations used in Sling Model

- **@Model:** Marks the class as a Sling Model; declares adaptables.
- **@Inject:** Generic injection (works for most cases).
- **@ValueMapValue:** Injects a property from the resource's ValueMap.
- **@ChildResource:** Injects a child resource (or model adapted from it).
- @OSGiService – Injects an OSGi service.
- @Self – Injects the adaptable itself (Resource / Request).
- @SlingObject – Injects Sling-related objects (ResourceResolver, Request, Response).
- @ScriptVariable – Injects HTL global objects (currentPage, pageProperties).
- @RequestAttribute – Injects request attributes.
- @PostConstruct – Method called after injection is complete.

- @Named, @Optional, @Default – modifiers for the above.

Q17. Sling Servlets

Servlets in AEM are typically registered as OSGi services and resolved by Sling either by path or by resource type.

- Extend SlingSafeMethodsServlet (read-only, GET) or SlingAllMethodsServlet (read/write).
- Override doGet / doPost / doPut / doDelete.

Two registration styles:

```
// 1. By resource type (recommended) @Component(service = Servlet.class)
@SlingServletResourceTypes(     resourceTypes = "proj/components/page",     methods       =
HttpConstants.METHOD_GET,     selectors     = "json",     extensions    = "json") public
class PageJsonServlet extends SlingSafeMethodsServlet { ... }  // 2. By path (use only when
necessary) @Component(service = Servlet.class,            property = {
"sling.servlet.paths=/bin/proj/data",              "sling.servlet.methods=GET"
}) public class DataServlet extends SlingSafeMethodsServlet { ... }
```

Q18. Client Libraries – properties (allowProxy, dependencies, embed)

A clientlib is a cq:ClientLibraryFolder node containing JS and CSS organized for delivery.

- **categories:** String / String[] – logical name(s) used to include the clientlib via <sly data-sly-call="${clientlib.all @ categories='proj.base'}"/>.
- **dependencies:** Other clientlib categories that must be loaded BEFORE this one (does not merge code, just orders include).
- **embed:** Other clientlib categories whose code is merged INTO this clientlib's output file (reduces HTTP requests).
- **allowProxy:** Boolean. When true, clientlib located in /apps can be served via /etc.clientlibs/<path>.js (recommended, keeps /apps protected).
- Files: css.txt and js.txt list the order of resources, plus the actual .js / .css / .less files.

Q19. OSGi

OSGi (Open Service Gateway initiative) is a dynamic module system for Java. AEM is built on Apache Felix (an OSGi container).

- Bundle – a JAR with OSGi metadata in MANIFEST.MF (Bundle-SymbolicName, Bundle-Version, Import-Package, Export-Package).
- Service – Java interface implementation registered in the service registry.
- Configuration – via OSGi config admin (config files under /apps/<proj>/osgiconfig/config.<runmode>).
- Lifecycle states – INSTALLED, RESOLVED, STARTING, ACTIVE, STOPPING, UNINSTALLED.
- Web console at /system/console (bundles, services, configMgr, components).

Q20. Maven – build/deploy a project, generate archetype

- AEM projects are generated from the Adobe AEM Project Archetype (Maven archetype).

```
mvn -B archetype:generate \   -D archetypeGroupId=com.adobe.aem \   -D
archetypeArtifactId=aem-project-archetype \   -D archetypeVersion=<latest> \   -D
appTitle="My Site" -D appId="mysite" -D groupId="com.mysite"
```

Common build/deploy commands:

- **mvn clean install:** Compile, run tests, package.
- **mvn clean install -PautoInstallBundle:** Builds and pushes the bundle JAR to a running AEM via Felix Web Console.
- **mvn clean install -PautoInstallPackage:** Builds the content package and uploads/installs it via CRX Package Manager.
- **mvn clean install -PautoInstallPackagePublish:** Installs on the publish instance (port 4503).

Q21. Variable / environment settings in Maven

- Variables are defined in pom.xml under <properties> – e.g., <aem.host>localhost</aem.host>, <aem.port>4502</aem.port>.
- Profiles (<profile>) define environment-specific overrides – dev, stage, prod.
- Activate a profile with -P<name>, e.g., mvn clean install -Pprod.
- Sensitive values (passwords) come from ~/.m2/settings.xml using <server> entries.
- In AEM, environment-specific OSGi configs go under /apps/<proj>/osgiconfig/config.<runmode>/ (e.g., config.dev, config.author.prod).

Q22. DAM (Digital Asset Management)

- DAM is AEM's repository for digital assets stored under /content/dam.
- Each asset is a dam:Asset node with original rendition + auto-generated renditions, metadata, and references.
- Asset Compute Service / DAM Update Asset workflow processes uploaded assets (thumbnails, metadata extraction, FFmpeg for video).
- Supports metadata schemas, smart tags, asset collections, asset shares, AEM Assets Brand Portal.
- Authors can reference DAM assets in components through the pathfield/imagefield in dialogs.

Q23. Multifield

A multifield (granite/ui/components/coral/foundation/form/multifield) lets authors add a variable number of entries (e.g., a list of links).

Two flavors:

- **Simple multifield:** Single field repeated. Stored as a String[] property.
- **Composite multifield:** Multiple sub-fields per entry (composite="true"). Stored as child nodes (item0, item1, …) under a parent node.

In Sling Model, inject composite multifield as List<ChildModel> using @ChildResource on the parent node.

Q24. Any challenges you faced?

Common real-world challenges and how they were solved:

- Dispatcher cache invalidation issues – fixed by configuring the right /statfileslevel and using sling:vanityPath carefully.
- Sling Model not injecting values – mismatch between adaptable type or missing @ValueMapValue(name="jcr:title").
- Clientlibs not loading on publish – allowProxy=false and library not in /etc.clientlibs path; fixed by setting allowProxy=true.

- Editable template policy changes not reflecting on existing pages – had to re-publish the template.
- Performance issues – large queries without indexes; created Oak indexes.
- Replication failures – fixed by checking the replication agent transport user and ACLs on /bin/receive.

Q25. Maven build commands – mvn clean install -P autoInstallBundle

- **mvn clean:** Removes the target/ folder.
- **mvn install:** Builds the project, runs tests, installs artifacts to local repository.
- **mvn clean install:** Combined – clean + install.
- **-P autoInstallBundle:** Activates the profile that uploads only the bundle JAR to a running AEM instance (faster iteration during development).
- **-P autoInstallPackage:** Uploads the full content package (CRX) to the author instance.
- **-P autoInstallPackagePublish:** Same, but to the publish instance.
- **-P autoInstallSinglePackage:** Installs a single 'all' package containing both bundle and content (used with the all module in modern archetypes).

Section B – Your additional 24 questions

Q1. Author and Publisher

Same answer as Section A Q5 – the two main AEM instance types:

- **Author:** Editing environment for authors / content creators. Default port 4502. Sits behind firewall.
- **Publisher:** Public-facing instance that serves the live website. Default port 4503. Sits in DMZ behind dispatcher.
- Content moves from Author → Publish via replication (default agent /etc/replication/agents.author/publish).

Q2. Dispatcher

Dispatcher is Adobe's caching/load-balancing module that sits between the Apache/HTTPD web server and the AEM publish instance(s).

- Caches HTML, JS, CSS, images on the file system → drastically improves performance.
- Performs load balancing across multiple publishers.
- Security – filters request URLs, methods, selectors, suffixes via the filter section.
- Invalidates cache when activation happens (via /dispatcher/invalidate.cache).
- Config files: dispatcher.any (farms, filters, cache rules).
- In AEMaaCS, configured via the Dispatcher SDK; rules are validated by Cloud Manager before deploy.

Q3. JCR

(Same concept as Section A Q6.) Java Content Repository – hierarchical, transactional content store implemented in AEM by Apache Jackrabbit Oak.

- Tree of Nodes; nodes hold Properties (name + value).
- Primary node type defines structure; mixin types add facets.

- APIs: Session, Node, Property, QueryManager.
- Supports observation (listeners), versioning, search.

Q4. Sling

Apache Sling is the RESTful web framework AEM is built on. It maps HTTP requests to resources in the JCR.

- URL → Resource resolution via ResourceResolver and resource type.
- Request decomposition: path, selectors, extension, suffix (e.g., /content/site/page.print.a4.html/foo/bar).
- Scripts (HTL/JSP) are resolved through the script resolution mechanism (resourceType + selector + extension).
- Provides Sling Models, Sling Servlets, Sling Events, Sling Scheduler, ResourceResolverFactory, etc.

Q5. High-level AEM architecture

- Layer 1 – User / Browser.
- Layer 2 – CDN (optional, e.g., Akamai / Fastly / AEMaaCS CDN).
- Layer 3 – Apache HTTPD + Dispatcher module (caching + filtering + load-balancing).
- Layer 4 – AEM Publish instance(s) – Sling + Felix (OSGi) + Jackrabbit Oak.
- Layer 5 – AEM Author instance – same stack but used for authoring.
- Replication moves content Author → Publish.
- Persistence: TarMK (file system) for single-node or MongoMK (MongoDB) / Document NodeStore (Cloud) for cluster, with DataStore (S3, Azure Blob, file) for binaries.

Q6. OSGi

Already covered in Section A Q19. Summary:

- Module system: code packaged as bundles (JARs).
- Service-oriented: components register / consume services.
- Dynamic: bundles/services can be installed, started, stopped, updated at runtime.
- AEM uses Apache Felix as its OSGi container.

Q7. OSGi configuration annotations

Modern AEM uses OSGi DS (Declarative Services) R6+ annotations:

- @Component – declares an OSGi component (often with service = Xxx.class).
- @Service / service attribute – registers as a service.
- @Activate, @Modified, @Deactivate – lifecycle methods.
- @Reference – injects another OSGi service.
- @ObjectClassDefinition (OCD) + @AttributeDefinition – type-safe configuration interface.
- @Designate – binds OCD to a component.

```
@ObjectClassDefinition(name="My Service Config") public @interface Config {
@AttributeDefinition(name="API URL")     String apiUrl() default "https://api.example.com";
}  @Component(service = MyService.class) @Designate(ocd = Config.class) public class
MyServiceImpl implements MyService {     @Activate @Modified     protected void
activate(Config cfg) { this.url = cfg.apiUrl(); } }
```

Q8. Why must the HTML file have the same name as the component node?

Sling resolves the script for a component using its resource type. By default, Sling looks for a file named <componentName>.html inside the component folder.

- If component node is /apps/proj/components/title, Sling looks for /apps/proj/components/title/title.html.
- This is the default 'main' script when no selector / extension match a more specific script.
- Other scripts (e.g., title.json.html or POST.html) handle other selectors / methods.
- If the file name doesn't match, Sling falls back to super type, or returns an error if none exists.

Q9. Components and Client Libraries

Components produce markup; clientlibs provide their JS/CSS.

- Components → /apps/<proj>/components, contain HTL + dialog + model.
- Clientlibs → /apps/<proj>/clientlibs, contain CSS / JS / less / icons.
- Components reference clientlibs by their category, included on a page typically at the page-component level.
- Best practice: each component has its own clientlib with category proj.components.<comp>; included via the page's main clientlib using 'embed' or 'dependencies'.

Q10. How to connect clientlib to a component

- Create a clientlib folder inside the component, e.g., /apps/proj/components/title/clientlibs/site.
- Set categories="proj.components.title", allowProxy=true.
- Add to the parent / page clientlib via the embed or dependencies property.
- Or include directly in HTL using <sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html" data-sly-call="${clientlib.all @ categories='proj.components.title'}"/> (but page-level inclusion is preferred).

Q11. How to connect a Sling Model with a component

- Write the Sling Model class with @Model(adaptables = Resource.class, resourceType = "proj/components/title").
- Mark with @Exporter for JSON exposure if needed.
- In HTL: data-sly-use.bean="com.proj.models.TitleModel" then access ${bean.title}.
- Make sure the bundle is OSGi-installed and the Sling Models package is scanned (set in bundle's MANIFEST.MF via Sling-Model-Packages header – handled automatically by the AEM archetype).

Q12. Adaptables, cq:dialog

Adaptables (in Sling Models) define what objects the model can be adapted from:

- **Resource:** Use when the model only needs JCR data – best for component models. Lightweight.
- **SlingHttpServletRequest:** Use when the model needs request data – e.g., request parameters, request attributes, current page, current style. Used for components that change behavior based on request.

cq:dialog – covered in Section A Q10; it's the Touch UI authoring dialog node.

Q13. How to register a servlet

Two ways (already shown in Section A Q17):

- By resource type (preferred): @SlingServletResourceTypes(resourceTypes = ..., methods = ..., selectors = ..., extensions = ...).
- By path: @SlingServletPaths("/bin/proj/api").

Always register as an OSGi service of type javax.servlet.Servlet (or jakarta.servlet.Servlet in newer AEMaaCS).

Q14. Two methods in Servlet interface

The javax.servlet.Servlet interface itself defines:

- init(ServletConfig)
- service(ServletRequest, ServletResponse)
- destroy()
- getServletConfig()
- getServletInfo()

In Sling, you don't usually implement the interface directly – you extend:

- **SlingSafeMethodsServlet:** Provides doGet, doHead, doOptions, doTrace – read-only.
- **SlingAllMethodsServlet:** Adds doPost, doPut, doDelete – read/write.

Most commonly overridden methods: doGet(SlingHttpServletRequest req, SlingHttpServletResponse resp) and doPost(...).

Q15. Static vs Editable templates

- **Static template:** Older approach. Defined under /apps/<proj>/templates. Page structure hard-coded in HTL / JSP. Changes require code deploy. Page is decoupled from template after creation.
- **Editable template:** Newer (6.2+). Defined under /conf/<proj>/settings/wcm/templates. Authors define structure / policies through UI. Page maintains a live connection (cq:template) and inherits structural changes.
- Editable templates are the recommended approach today. AEMaaCS strongly encourages editable templates.

Q16. Initial content, Structure, Layout (editable template modes)

- **Structure:** Defines locked components (header, footer) and the unlocked area where authors can drop components. Locked = page authors cannot edit. Unlocked area = parsys / container.
- **Initial Content:** Pre-populated components on every new page created from this template. Authors CAN modify these on individual pages, but they appear on every new page by default.
- **Layout:** Define responsive layout (12-column grid) for breakpoints (mobile/tablet/desktop).
- **Policies:** Per component, define allowed values / configuration – e.g., for the text component which RTE plugins are enabled.

Q17. How to create a clientlib

- Right-click /apps/<proj>/clientlibs → Create → Create Node.
- Node type: cq:ClientLibraryFolder. Name: e.g., clientlib-site.

- Add properties: categories (String[]), allowProxy (Boolean), and optionally dependencies / embed (String[]).
- Inside the folder, create css.txt and js.txt listing files in order, and add css/ and js/ subfolders with the actual files.

```
clientlib-site/ ├── .content.xml          (categories, allowProxy) ├── css.txt
(#base=css\n  styles.css) ├── css/ │   └── styles.css ├── js.txt               (#base=js\n
main.js) └── js/     └── main.js
```

Q18. allowProxy, categories, dependencies, embed

- **categories:** Logical name(s). Used to include the clientlib in HTL or via dependencies/embed of other clientlibs.
- **allowProxy:** If true, the clientlib (under /apps) is served via /etc.clientlibs/. Keeps /apps protected from public access. Always set to true in AEMaaCS.
- **dependencies:** Other clientlib categories that must load BEFORE this one. They are included as separate <script>/<link> tags.
- **embed:** Other clientlib categories whose code is MERGED into this clientlib's output (one combined file – fewer HTTP requests).

Q19. Page policy

A page policy is the configuration of an editable template's page component (jcr:content of the structure node). It defines:

- Which clientlib categories load on every page using this template.
- Default page properties (e.g., default redirect behavior, social tags).
- Webpage meta-data defaults.

Policies in general (per component) live under /conf/<proj>/settings/wcm/policies and are versioned/reusable. Page policy is just the policy of the root page component.

Q20. Blueprint, Live Copy, Language Copy

- **Blueprint:** A 'master' site used as the source. Defined as a blueprint configuration under /etc/msm/. Holds the original content.
- **Live Copy:** A copy of the blueprint (or any source page). Maintains a live relationship – changes in source can roll out to live copy. Each component has 'inheritance' that can be cancelled per field.
- **Language Copy:** A specific kind of copy used for translation – content is copied and sent to a translation provider (machine or human). Maintained per locale under /content/<site>/<locale>.
- Rollout = pushing changes from blueprint to live copies.
- Synchronize = manual rollout for a specific page.

Q21. Multi-Site Manager (MSM)

MSM is the AEM framework for managing multiple sites that share content/structure, often used for:

- Multi-region / multi-brand sites that share a common base.
- Multi-language / localized variations of a single site.

Key concepts: Blueprint, Live Copy, Rollout configuration, Inheritance, Suspend/Cancel inheritance, Detach live copy.

Rollout configs determine what happens when a blueprint is changed – e.g., Standard rollout copies content; Activate after Rollout also publishes.

Q22. Can we create a Language Copy from a Live Copy?

Yes. AEM supports a 'translation-aware live copy' – you can create a Live Copy and configure it as a Language Copy by:

- Creating a Live Copy with the appropriate rollout configuration.
- Or, creating a Language Copy from the Sites console (References Rail → Language Copies → Create & Translate).

Best practice in modern AEM: keep a master in a single language (e.g., /content/site/language-masters/en), create Live Copies for each region, and Language Copies within each region for additional languages. Language Copies and Live Copies can be combined – the 'Catalyst' / 'Translation Integration Framework' handles this scenario.

Q23. Experience Fragment vs Content Fragment

- **Experience Fragment (XF):** A reusable, designed group of components that include layout and styling. Used for marketing experiences (e.g., a hero block, a footer) that need to be reused across pages, sites, and channels (email, Facebook, Pinterest). Lives under /content/experience-fragments/.
- **Content Fragment (CF):** Structured, channel-agnostic CONTENT only (no layout). Defined by a Content Fragment Model that specifies fields and types. Used for headless delivery via GraphQL or Sling Model exporter. Lives under /content/dam/<path>.

Rule of thumb: Experience Fragments = designed content; Content Fragments = pure content for any channel.

Q24. Modules in Maven archetype folder structure

The Adobe AEM Project Archetype generates a multi-module Maven project. Typical modules:

- **core:** Java code – Sling Models, Servlets, OSGi services, filters. Compiles to an OSGi bundle (.jar).
- **ui.apps:** Code that goes into /apps – components, templates structures, clientlibs, OSGi configs. Packaged as a content package.
- **ui.content:** Sample / required content under /content – sample pages, tags, languages.
- **ui.config:** OSGi runtime configurations (separate from ui.apps in modern archetype, especially for AEMaaCS).
- **ui.frontend:** Webpack-based front-end build (TypeScript, SCSS) → compiled output is copied into ui.apps as clientlibs.
- **ui.tests:** End-to-end tests (UI tests, Hobbes / WDIO).
- **it.tests:** Integration tests run against a live AEM.
- **dispatcher:** Apache + Dispatcher configuration files (AEMaaCS structure).
- **all:** Aggregator – produces a single 'all' container package that includes ui.apps + ui.content + ui.config + core bundle. This is what gets deployed via Cloud Manager.
- **analyse (optional):** Static analysis to fail builds with policy violations.

— End of document —